

Capture the Flag 5: Make the System Interactive

Goal: Transform your Student Directory into an interactive application where administrators can favorite students, edit information, and remove records using Flutter interactions and state management.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

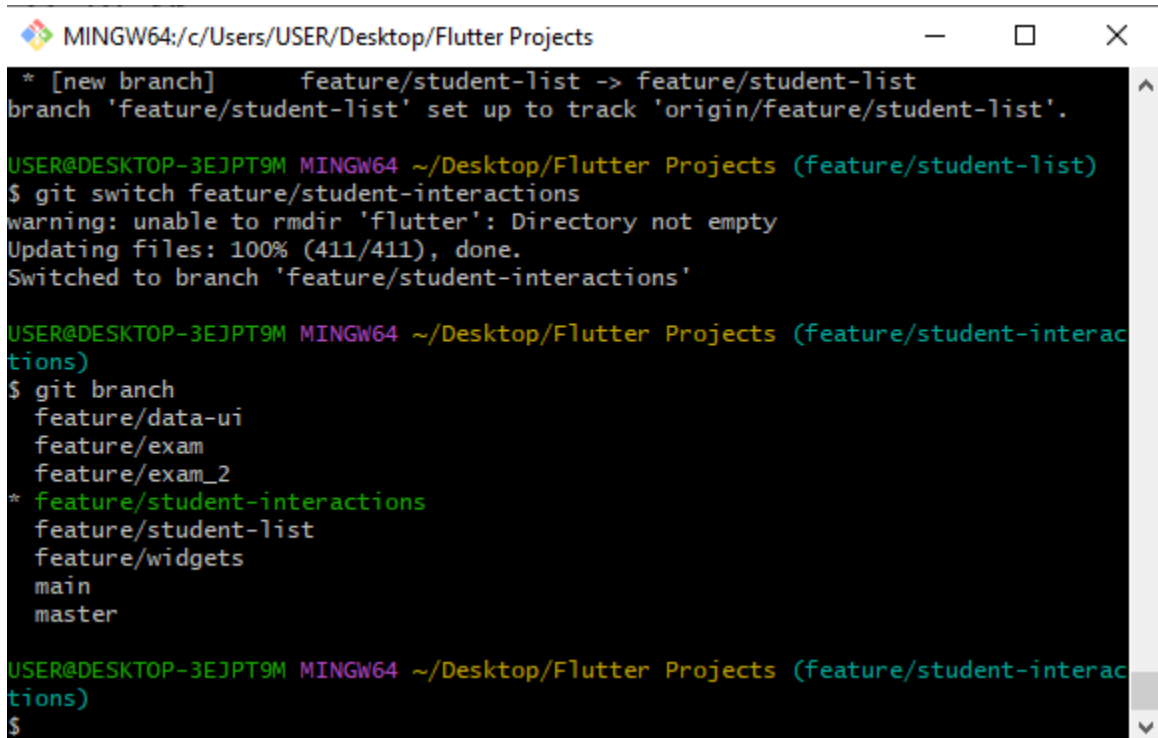
Objective: Create a new feature branch before adding interactions.

Create:

`feature/student-interactions`

Screenshot:

- Terminal showing your current branch



```
MINGW64:/c/Users/USER/Desktop/Flutter Projects
* [new branch]      feature/student-list -> feature/student-list
branch 'feature/student-list' set up to track 'origin/feature/student-list'.

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/Flutter Projects (feature/student-list)
$ git switch feature/student-interactions
warning: unable to rmdir 'flutter': Directory not empty
Updating files: 100% (411/411), done.
Switched to branch 'feature/student-interactions'

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/Flutter Projects (feature/student-interac
tions)
$ git branch
  feature/data-ui
  feature/exam
  feature/exam_2
* feature/student-interactions
  feature/student-list
  feature/widgets
  main
  master

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/Flutter Projects (feature/student-interac
tions)
$
```

//Nag buhat-buhat og unag branch miss nya wa diay nakahuman sa past flag activity.

🚩 Flag 1 — Wake Up the Buttons

Add at least two buttons inside each Student Card.

Example actions:

- Favorite
- Edit

Objective: Make your Student Card respond to button presses using `onPressed`.

Right now your buttons are only decorations.

Research how Flutter's `onPressed` works.

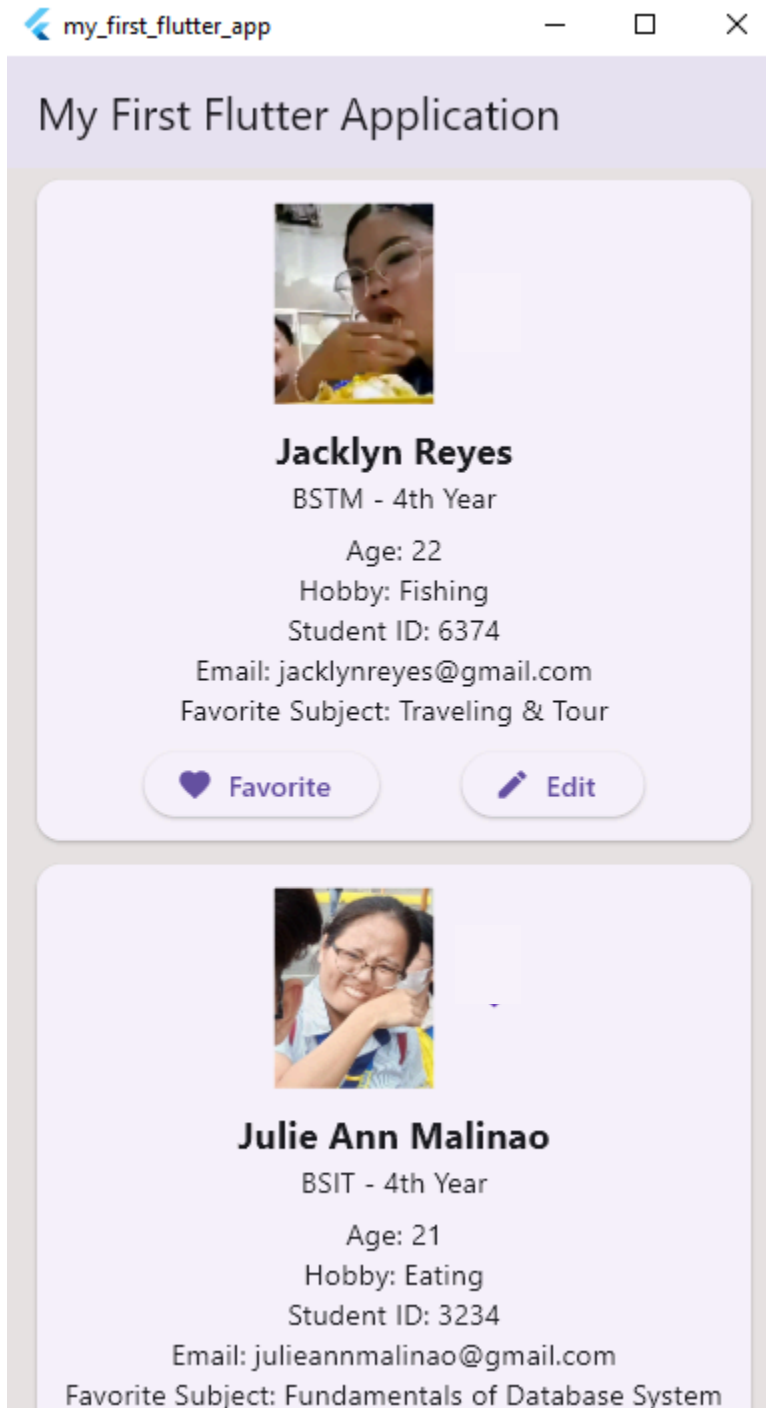
The buttons do not need to perform their final behavior yet. They simply need to respond when pressed.

Test

When a button is pressed, your app should visibly react in some way (such as printing a message or showing a temporary notification).

Screenshot:

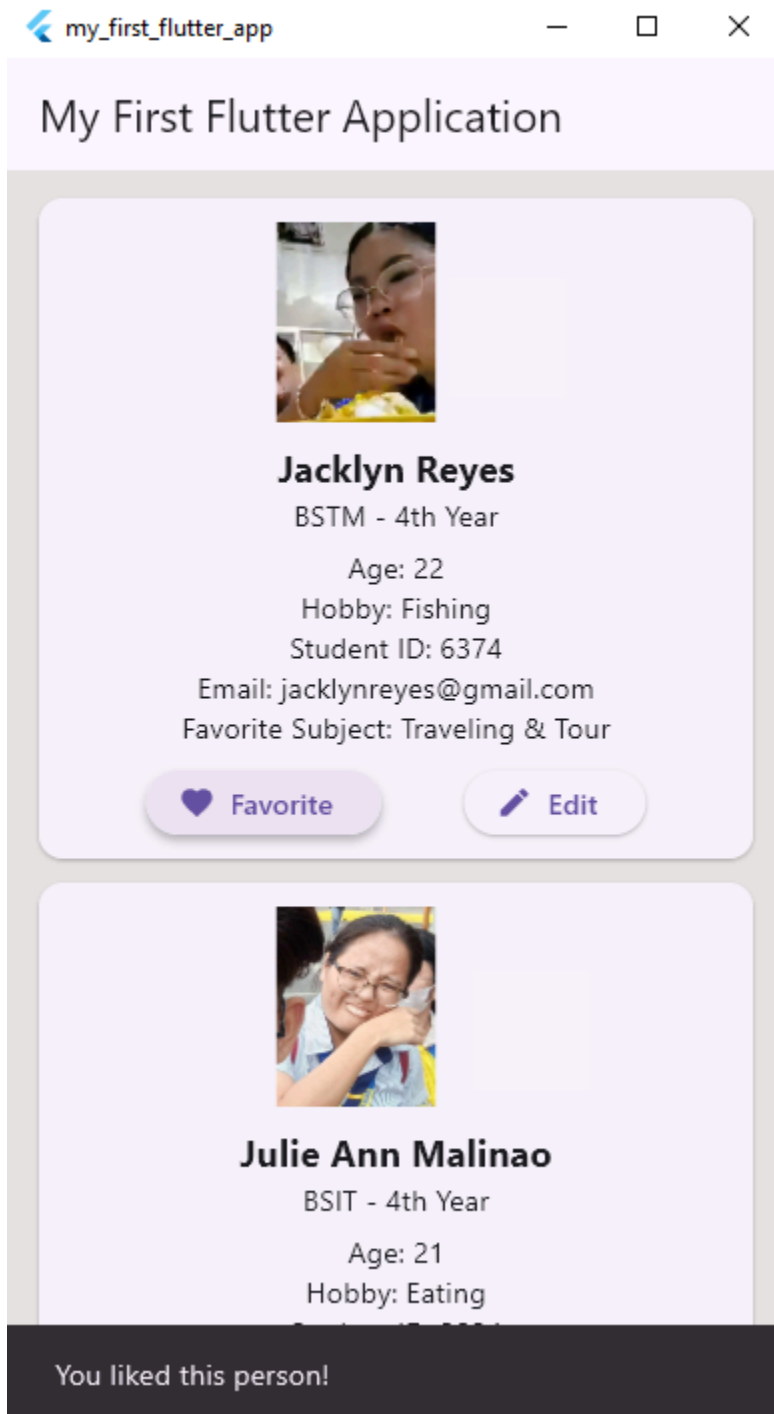
- Student Card with buttons



- `onPressed` inside your code

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
  children: [  
    ElevatedButton.icon(  
      onPressed: () => toggleFavorite(context, student),  
      icon: const Icon(Icons.favorite),  
      label: const Text('Favorite'),  
    ), // ElevatedButton.icon  
    ElevatedButton.icon(  
      onPressed: () => showEditDialog(context, student),  
      icon: const Icon(Icons.edit),  
      label: const Text('Edit'),  
    ), // ElevatedButton.icon  
  ],  
) // Row
```

- Button reacting or terminal message



🚩 Flag 2 — Touch Anywhere

Objective: Make the entire Student Card tappable using `onTap`.

Buttons aren't the only interactive elements.

Research Flutter's `onTap`.

Wrap your Student Card so tapping anywhere on the card triggers an interaction.

Test

The user should be able to:

- Tap the Favorite button
- Tap the Edit button
- Tap anywhere else on the Student Card

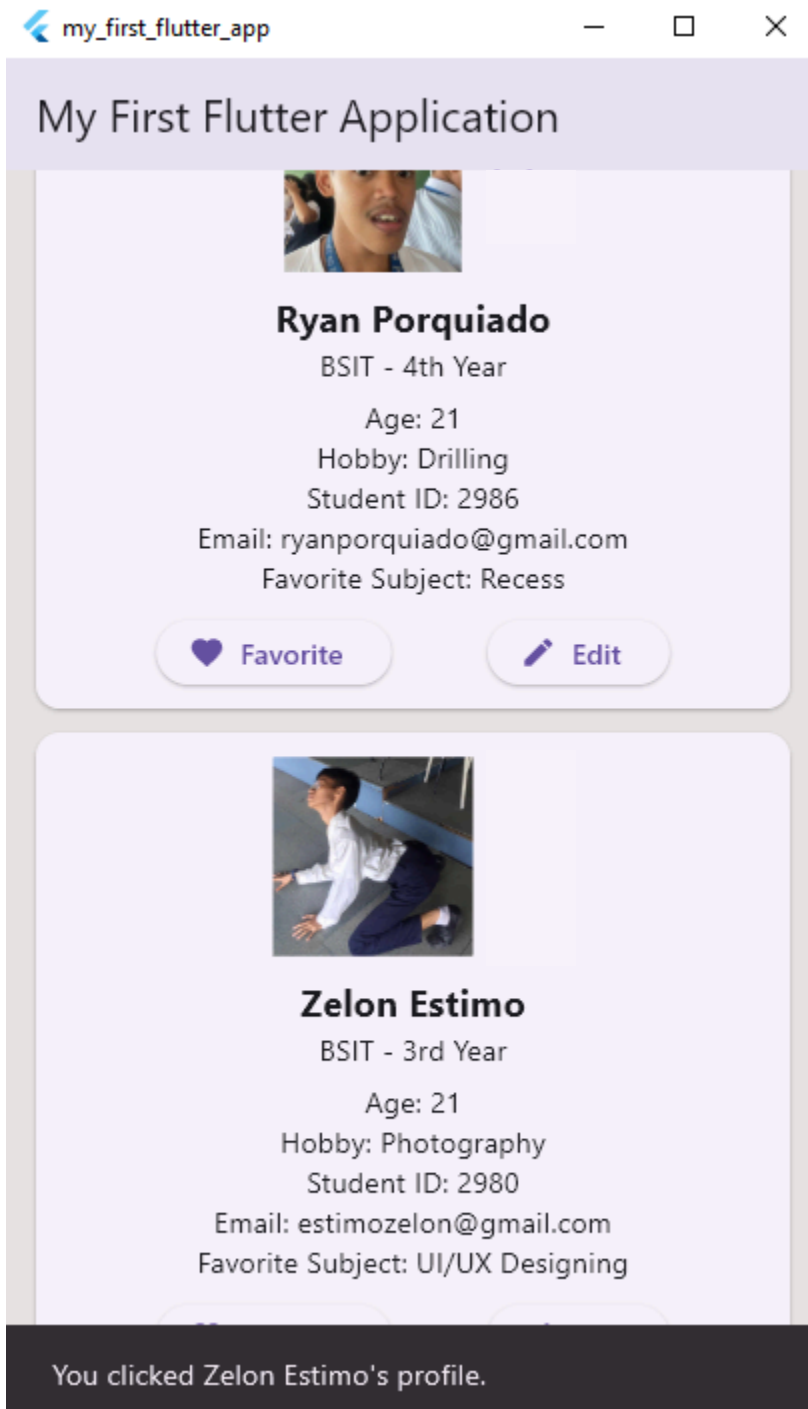
Each interaction should be recognized separately. (One function for the 2 buttons, another for the Student Card)

Screenshot:

- `onTap` in your code

```
onTap: () {  
  ScaffoldMessenger.of(context).showSnackBar(  
    SnackBar(  
      content: Text("You clicked ${student.name}'s profile."),  
      duration: const Duration(seconds: 1),  
    ), // SnackBar // SnackBar  
  );  
},
```

- Student Card being tapped



My First Flutter Application



Ryan Porquiado

BSIT - 4th Year

Age: 21

Hobby: Drilling

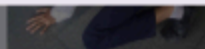
Student ID: 2986

Email: ryanporquiado@gmail.com

Edit Ryan Porquiado

Edit form coming soon.

Close



Zelon Estimo

BSIT - 3rd Year

Age: 21

Hobby: Photography

Student ID: 2980

Email: estimozelon@gmail.com

Favorite Subject: UI/UX Designing

♥ Favorite

✎ Edit

My First Flutter Application

Leonardo Cajés Jr.

BSTM - 4th Year

Age: 21

Hobby: Twerking

Student ID: 2988

Email: leonardocajes@gmail.com

Favorite Subject: Research



Favorite



Edit



Ryan Porquiado

BSIT - 4th Year

Age: 21

Hobby: Drilling

Student ID: 2986

Email: ryanporquiado@gmail.com

Favorite Subject: Recess

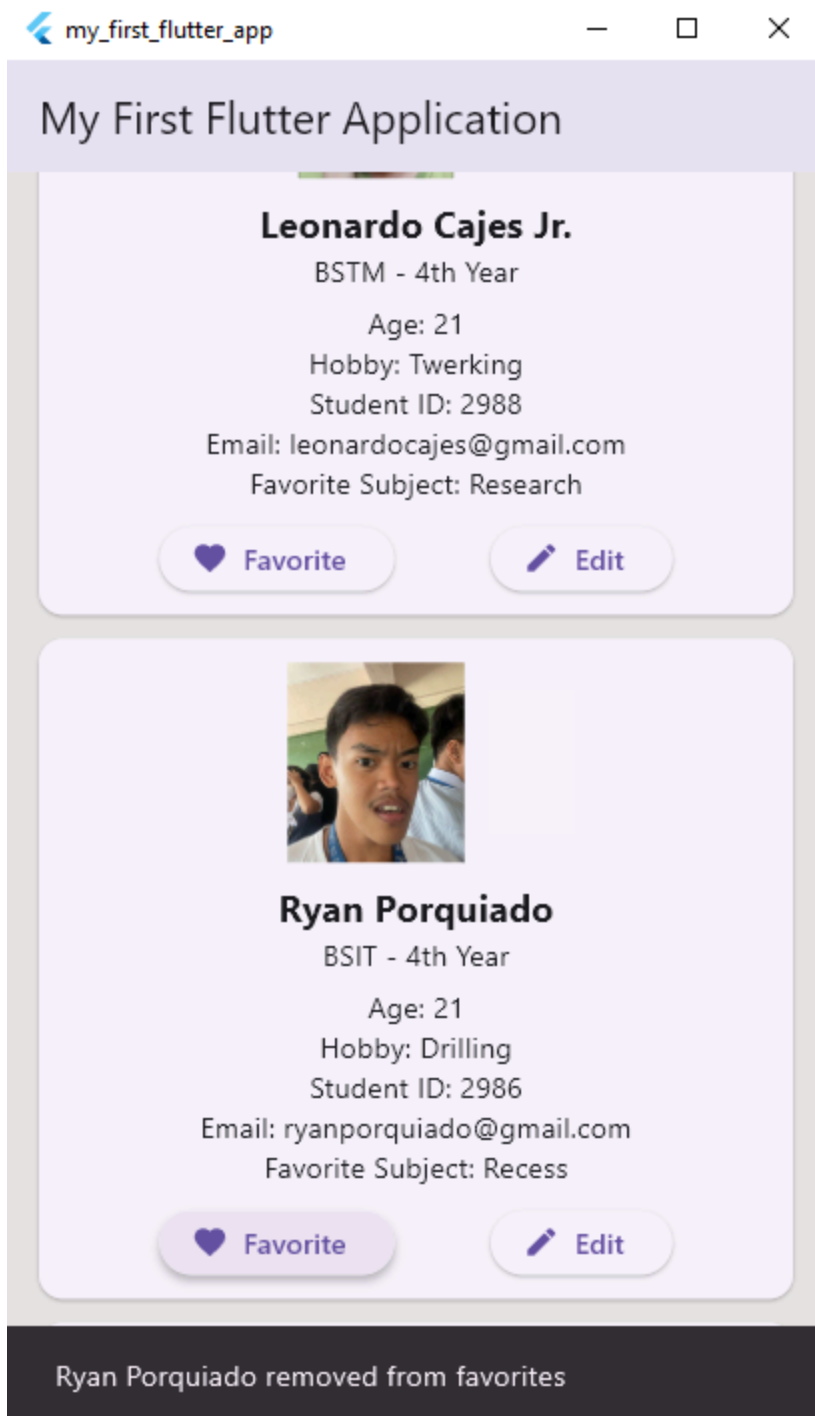


Favorite



Edit

You liked this person!



🚩 Flag 3 — Discover `setState`

Objective: Update the screen immediately using `setState`.

So far your app reacts. But does the UI actually change?

Research Flutter's `setState`.

Use it so pressing a button immediately updates something visible on the screen.

Examples include:

- Changing text
- Changing an icon
- Updating a counter

The important part is understanding that `setState` tells Flutter to rebuild the UI.

Test

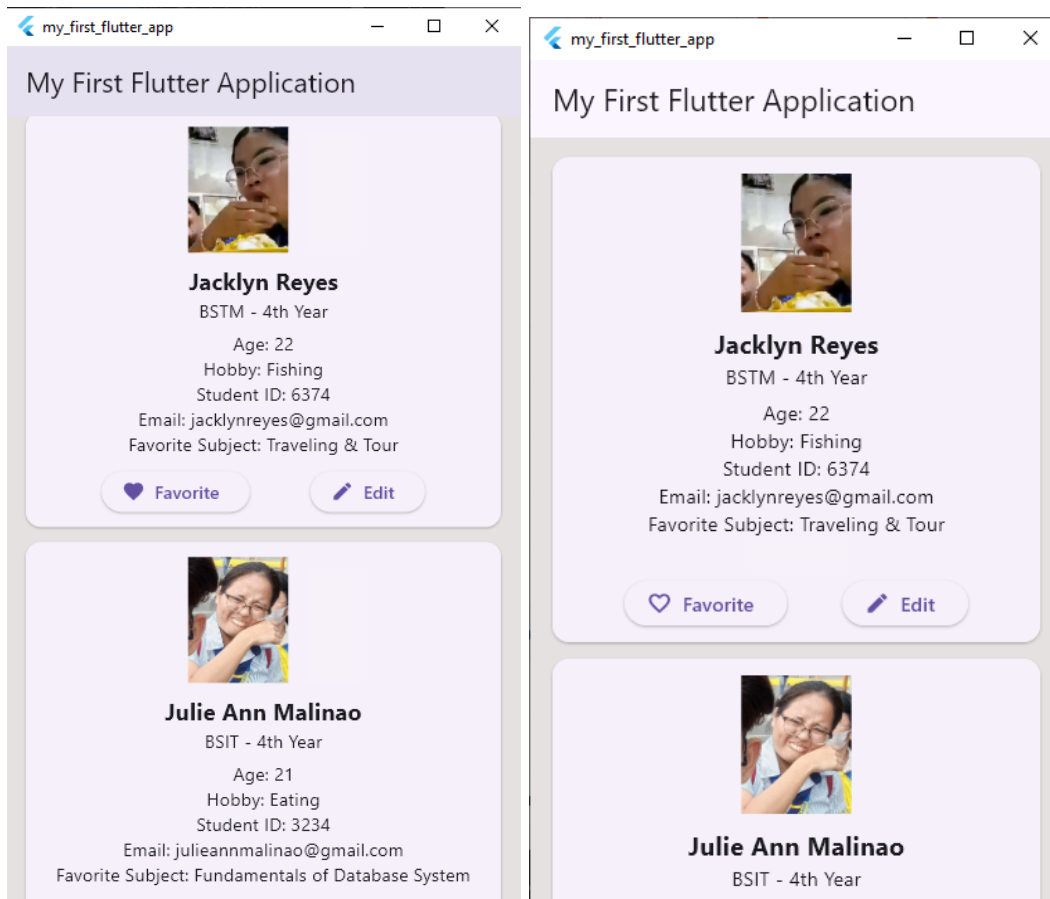
The screen should visibly change immediately after pressing the button.

Screenshot:

- `setState` in your code

```
setState(() {  
  student.isFavorite = !student.isFavorite;  
});  
  
print("${student.name} Favorite pressed");
```

- Before and after interaction



🚩 Flag 4 — Favorite Students

Objective: Let administrators mark students as favorites.

Schools often need to highlight important records.

Add a favorite feature that changes a student's appearance when marked.

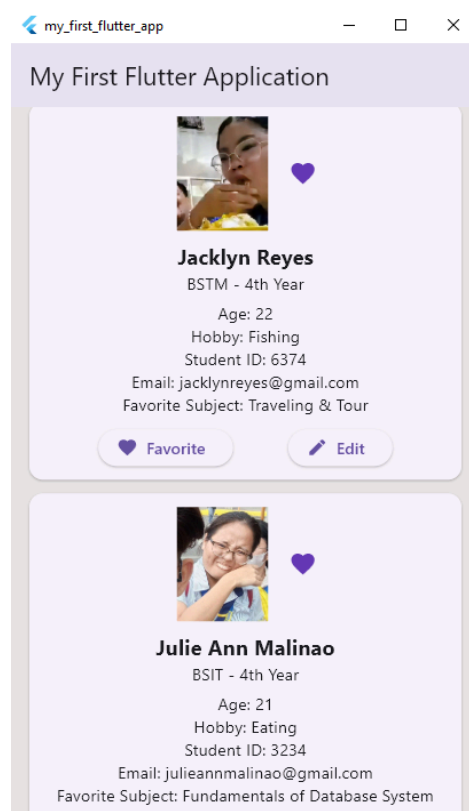
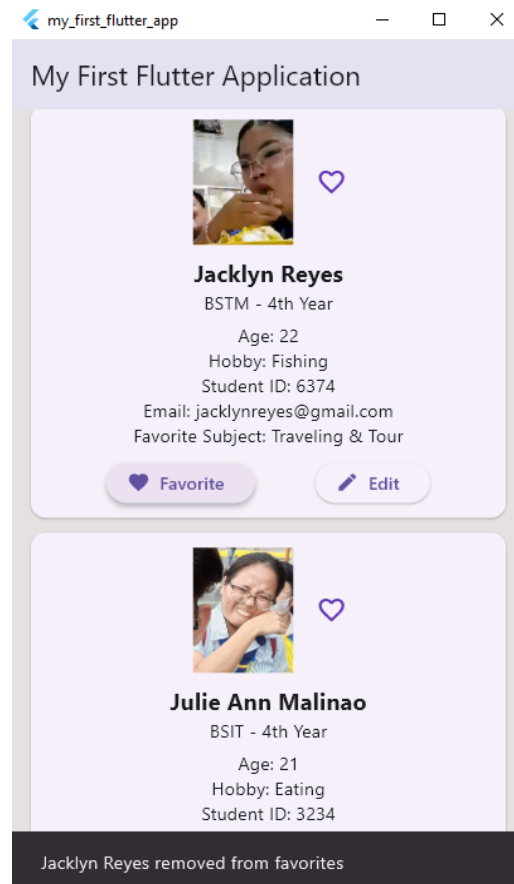
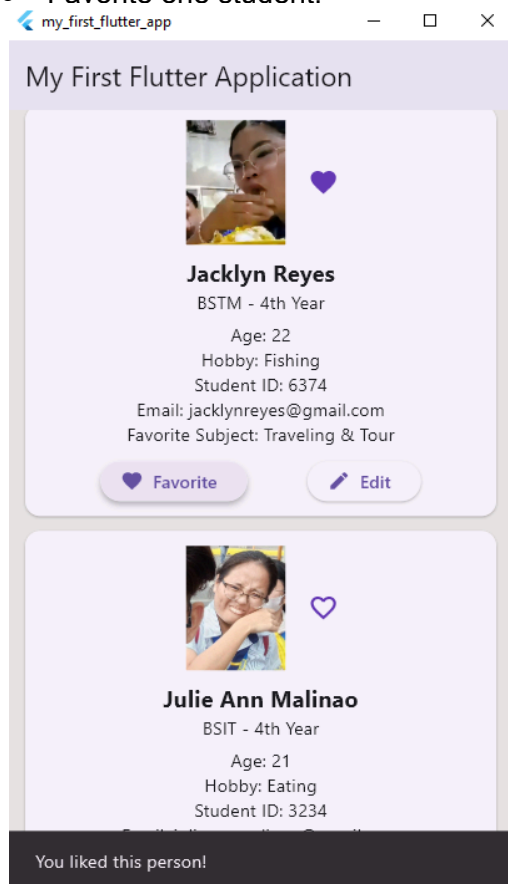
Possible visual changes include:

- Filled heart/star
- Different card color
- Favorite label

Each student's favorite status should work independently.

Test

- Favorite one student.



Flag 5 — Remove a Student

Objective: Delete a student from the directory.

Research how to remove an item from a Dart `List`.

Add a Delete action that removes a student from your list.

The UI should immediately update after deletion.

Test

Before:

- Student A
- Student B
- Student C

After deleting Student B:

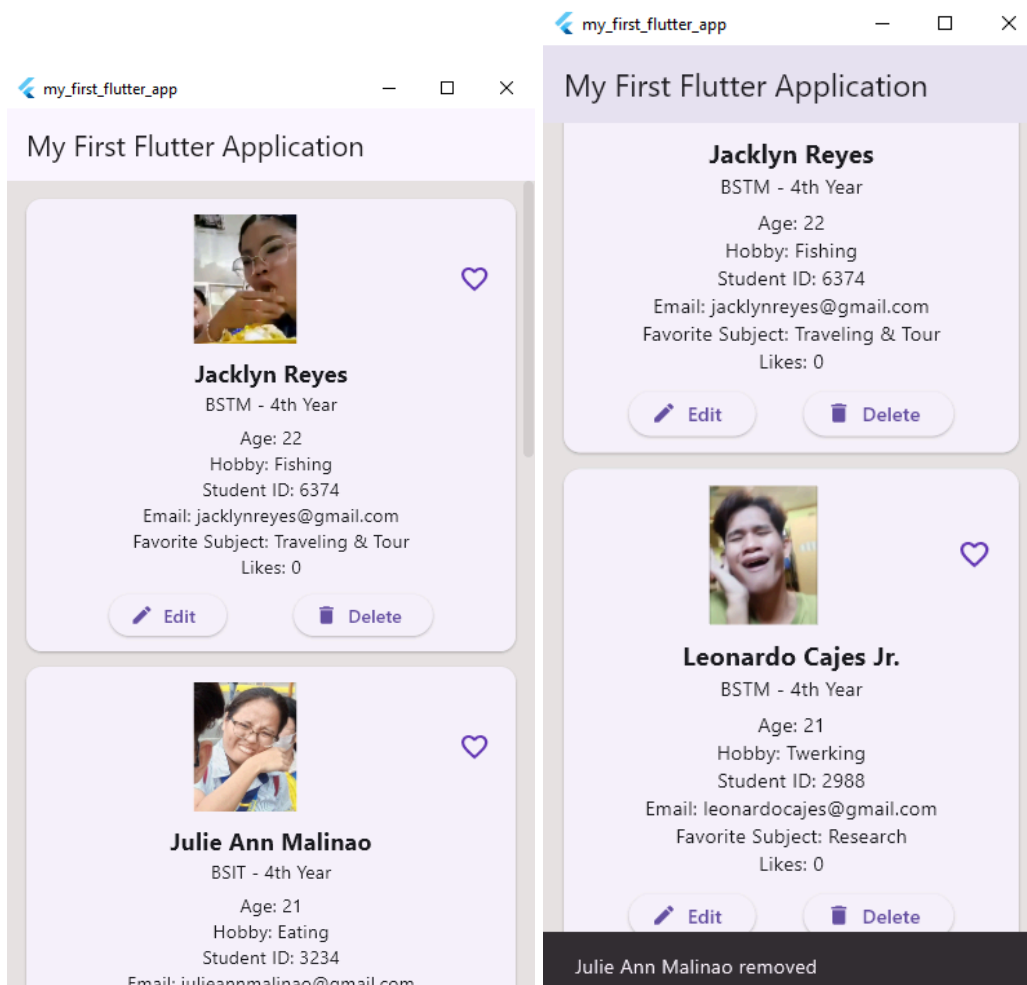
- Student A
- Student C

No empty spaces should remain.

Screenshot:

- Student before deletion

- Student after deletion



🚩 Flag 6 — Edit Trigger

Objective: Prepare your app for editing student information.

In the next CTF you'll build a full Edit Screen.

For now, create an Edit trigger.

Research simple ways Flutter can display temporary editing interfaces.

Examples include:

- `AlertDialog`

- **SnackBar**
- Bottom Sheet

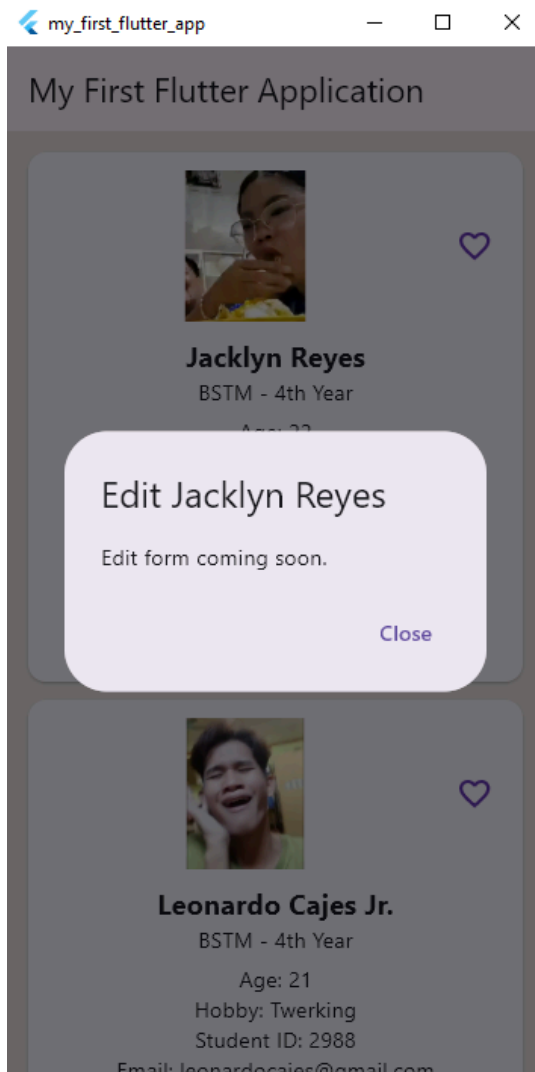
The goal is simply to launch an edit action, not build the complete editor yet.

Test

Pressing Edit should open your chosen interface.

Screenshot:

- Edit button



- Opened dialog, snackbar, or bottom sheet

```
void showEditDialog(BuildContext context, Student student) {  
  showDialog(  
    context: context,  
    builder: (context) {  
      return AlertDialog(  
        title: Text("Edit ${student.name}"),  
        content: const Text("Edit form coming soon."),  
        actions: [  
          TextButton(  
            onPressed: () => Navigator.pop(context),  
            child: const Text("Close"),  
          ), // TextButton  
        ],  
      ); // AlertDialog  
    },  
  );  
}
```

🚩 Flag 7 — Multiple Actions, Independent State

Objective: Make every Student Card manage interactions correctly.

This is your final interaction challenge.

Verify that every student behaves independently.

Example scenario:

- Favorite Student A.
- Delete Student C.
- Open Edit for Student B.

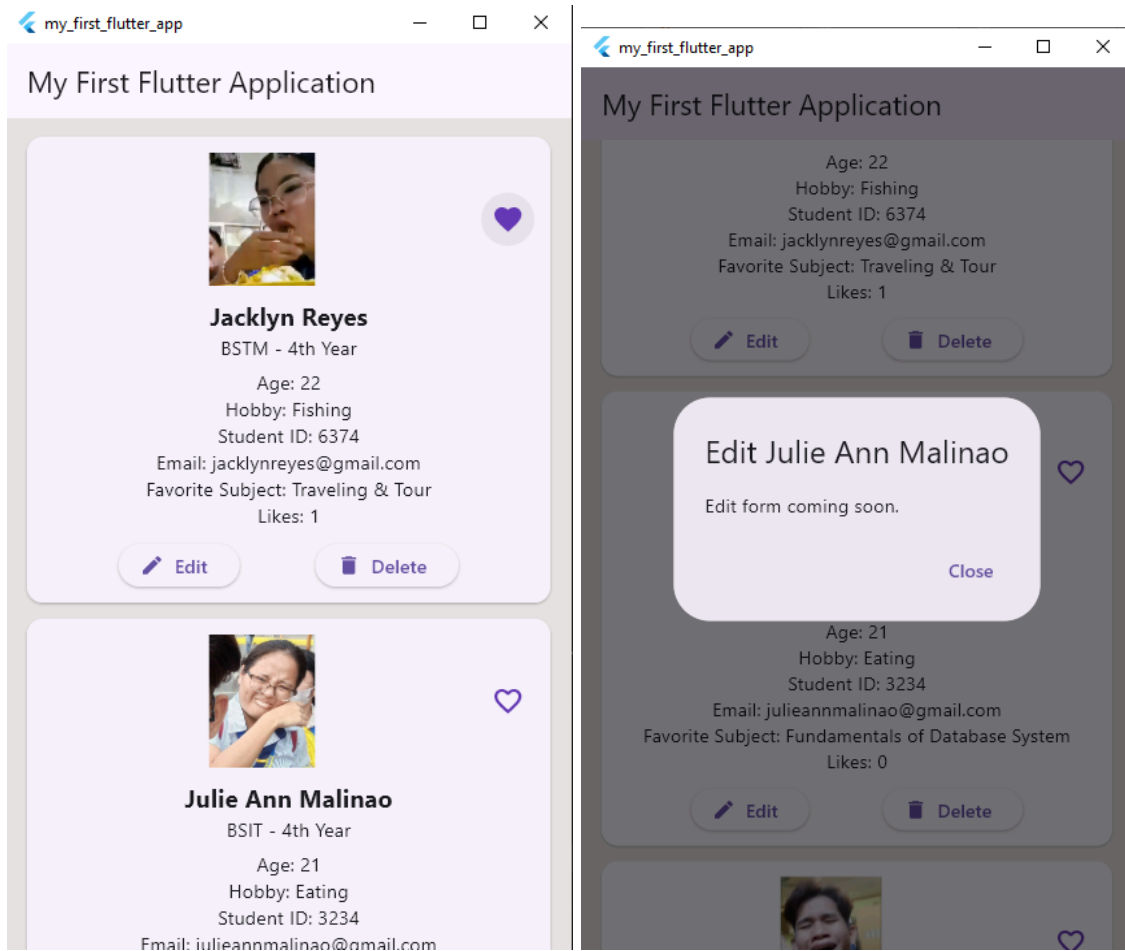
Your application should continue functioning correctly without affecting unrelated students.

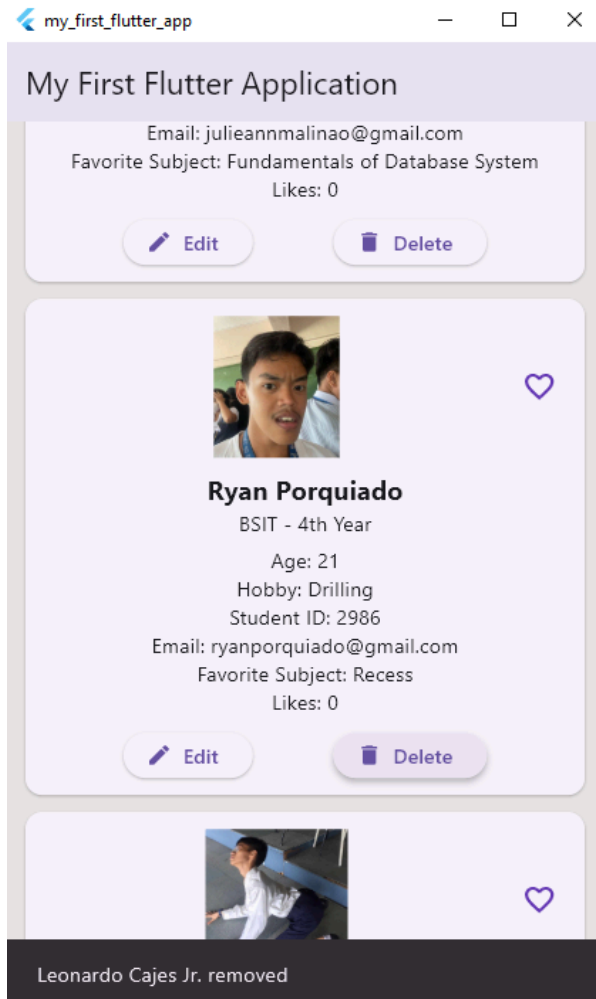
Test Checklist

- Favorite works independently.
- Delete updates the list immediately.
- Edit still opens correctly.
- Remaining students keep their own states.

Screenshot:

- Multiple interactions working together





P.S: (Miss ako gi tangtang ang Favorite na button kay since ang heart icon kay same ra silas Favorite button, bati kaayo kay redundant na huehueness.)

🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Interactive Student Directory.

Use a proper Git commit message.

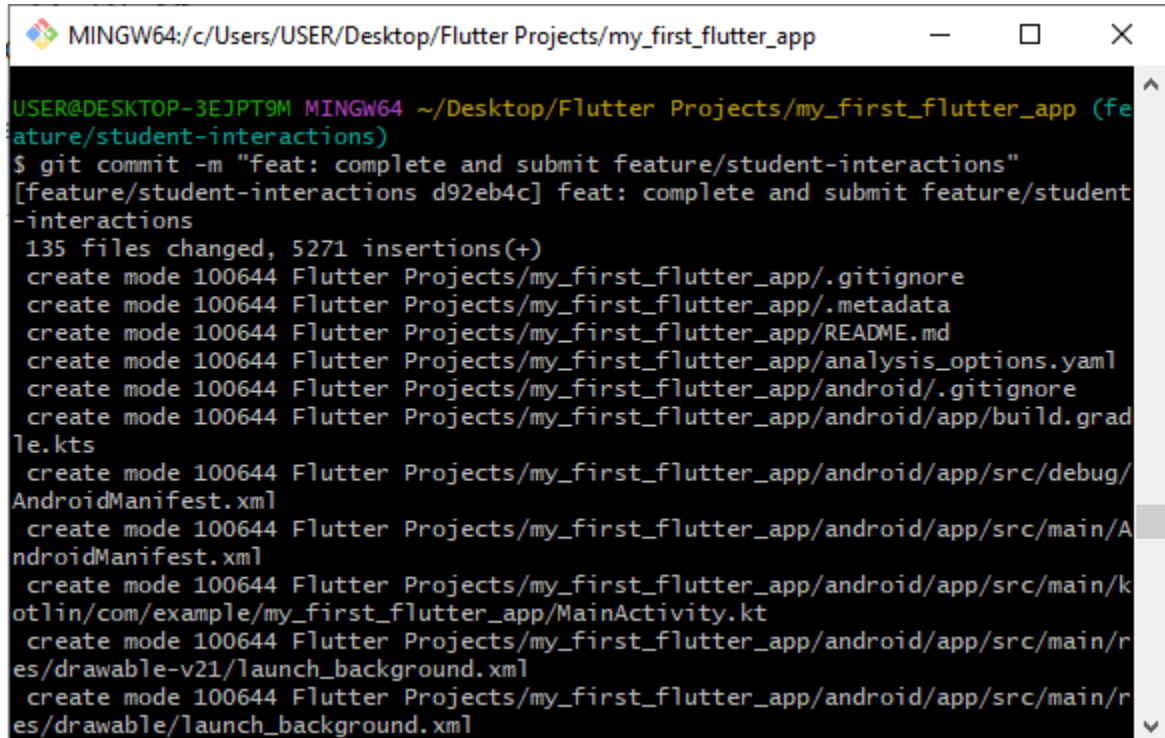
Branch:

`feature/student-interactions`

Screenshot:

- Successful commit

- Current branch
- Terminal showing the commit



A terminal window titled "MINGW64: c:/Users/USER/Desktop/Flutter Projects/my_first_flutter_app" showing a git commit command and its output. The output lists 135 files changed with 5271 insertions, including various Flutter project files like .gitignore, README.md, analysis_options.yaml, build.gradle, and AndroidManifest.xml.

```
MINGW64: c:/Users/USER/Desktop/Flutter Projects/my_first_flutter_app

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/Flutter Projects/my_first_flutter_app (feature/student-interactions)
$ git commit -m "feat: complete and submit feature/student-interactions"
[feature/student-interactions d92eb4c] feat: complete and submit feature/student-interactions
135 files changed, 5271 insertions(+)
create mode 100644 Flutter Projects/my_first_flutter_app/.gitignore
create mode 100644 Flutter Projects/my_first_flutter_app/.metadata
create mode 100644 Flutter Projects/my_first_flutter_app/README.md
create mode 100644 Flutter Projects/my_first_flutter_app/analysis_options.yaml
create mode 100644 Flutter Projects/my_first_flutter_app/android/.gitignore
create mode 100644 Flutter Projects/my_first_flutter_app/android/app/build.gradle.kts
create mode 100644 Flutter Projects/my_first_flutter_app/android/app/src/debug/AndroidManifest.xml
create mode 100644 Flutter Projects/my_first_flutter_app/android/app/src/main/AndroidManifest.xml
create mode 100644 Flutter Projects/my_first_flutter_app/android/app/src/main/kotlin/com/example/my_first_flutter_app/MainActivity.kt
create mode 100644 Flutter Projects/my_first_flutter_app/android/app/src/main/res/drawable-v21/launch_background.xml
create mode 100644 Flutter Projects/my_first_flutter_app/android/app/src/main/res/drawable/launch_background.xml
```